IT TICKET MANAGER

Итоговый отчёт по индивидуальной работе

Проектирование настольного приложения для регистрации и обработки IT-заявок

Параметр	Значение
Студент	Tcacenco Dmitrii
Группа	P2524r
Преподаватель	Natalia Gairunova
Дисциплина	Planificarea aplicațiilor Desktop
Вариант	10
Год	2026

Итоговый комплект этапов 1-5 и финальный отчёт этапа 6

Содержание

1. **Резюме проекта**
2. **Назначение, цель и границы**
3. **Пользователи и роли**
4. **Модули приложения**
5. **Жизненный цикл заявки**
6. **Планируемая архитектура и технологии**
7. **Спецификация требований SRS**
8. **UML-модель**
9. **Прототип интерфейса в Figma**
10. **Git и GitHub**
11. **Проверка согласованности задания**
12. **Заключение**
13. **Проверенные источники**

Отчёт объединяет материалы планирования, SRS, UML, прототипирования и версионирования. Реализация Java-приложения, подключение SQLite и фактическая SMTP-доставка email в рамках этого отчёта не выдаются за выполненные результаты: они описаны как планируемые функции будущей разработки.

1. Резюме проекта

IT Ticket Manager - проект настольного приложения для регистрации, обработки и контроля технических заявок в небольшой организации, колледже или учебной лаборатории. Система должна заменить разрозненные устные сообщения и переписки единым процессом с номером заявки, приоритетом, ответственным техником, статусом и историей действий.

В итоговом комплекте подготовлены: план проекта, спецификация требований SRS, Use Case и Activity диаграммы с исходниками PlantUML, шесть экранов Figma, Git-документация и публичный репозиторий. Все материалы согласованы вокруг двух обязательных ролей: Пользователь и IT-техник.

Показатель	Итог
Назначение	Регистрация и сопровождение технических заявок
Обязательные роли	Пользователь; IT-техник
Статусы	Открыта; В работе; Решена; Закрыта
Подтверждение	Событие пользователя, не отдельный статус
Закрытие	Доступно технику только после подтверждения
Email	Проектируемые события; SMTP-доставка не подтверждается
Артефакты	DOCX, PDF, UML, PlantUML, Figma, Git/GitHub

2. Назначение, цель и границы

2.1. Цель и задачи

Цель проекта - спроектировать понятное настольное приложение и согласованный комплект документации для последующей реализации. Проект решает задачи определения ролей и границ, описания жизненного цикла заявки, декомпозиции на модули, формирования требований, подготовки UML и прототипа Figma, а также организации материалов в Git.

- сократить риск потери обращений и неопределённости по ответственному исполнителю;
- дать пользователю прозрачный способ создания заявки и проверки результата;
- дать технику единую очередь, фильтры и историю обработки;
- зафиксировать проверяемые правила переходов и аудита.

2.2. Границы решения

Входит в проектирование	Не заявляется как выполненное
Роли, сценарии, статусы, требования, UML и UI-прототип	Работающее Java/JavaFX приложение
Планируемая модель SQLite и слоёв приложения	Реальная эксплуатация и реальные пользовательские данные

Проектируемые email-события	Фактическая отправка писем через SMTP
Документирование в DOCX/PDF и Git/GitHub	Готовая серверная или сетевая инфраструктура

3. Пользователи и роли

Базовая версия использует две обязательные роли. Администратор не включён в обязательный контур, поэтому не фигурирует в Use Case как самостоятельный актор.

Роль	Ответственность	Доступ
Пользователь	Создаёт заявку, смотрит свои заявки, добавляет комментарии, проверяет решение	Свои заявки и действия подтверждения/отказа
IT-техник	Просматривает очередь, принимает заявку, фиксирует решение, закрывает заявку	Очередь, обработка, статусы и закрытие после подтверждения

Правило подтверждения: после перевода заявки в «Решена» автор заявки подтверждает результат или возвращает её в «В работе» с причиной. Подтверждение записывается в историю как событие. Статус «Подтверждена» не вводится.

4. Модули приложения

Код	Модуль	Назначение
M1	Авторизация и сессия	Вход, выход, проверка роли, защита экранов
M2	Пользователи и права	Профиль, роль, активность, проверка разрешений
M3	Создание заявки	Форма, валидация, номер, статус «Открыта»
M4	Очередь техника	Поиск, фильтры, сортировка и принятие в работу
M5	Обработка заявки	Результат решения, статусы, комментарии
M6	Карточка и аудит	История событий, статусы, автор и исполнитель
M7	Уведомления	Подготовка email-событий и их статусов доставки

Модули описывают проектную декомпозицию. Они не означают, что отдельные программные компоненты уже реализованы.

5. Жизненный цикл заявки

Текущий статус	Действие	Новый статус	Кто	Условие
Нет	Создать	Открыта	Пользователь	Данные корректны
Открыта	Принять в работу	В работе	Техник	Назначен исполнитель
В работе	Зафиксировать решение	Решена	Техник	Результат обязателен
Решена	Подтвердить	Без смены статуса	Автор	Записывается событие
Решена	Отклонить	В работе	Автор	Причина обязательна
Решена	Закрыть	Закрыта	Техник	Есть подтверждение автора

Ключевое ограничение: закрытие заявки нельзя выполнить до подтверждения результата пользователем. Если пользователь отказывает, система сохраняет причину и возвращает заявку в «В работе».

6. Планируемая архитектура и технологии

Для будущей реализации предлагается слоистая архитектура: JavaFX отвечает за представление, прикладной сервис - за бизнес-правила и права, слой доступа - за работу с SQLite через JDBC. Такое разделение упрощает тестирование и позволяет изменять технические детали без переписывания интерфейса.

Компонент	Планируемое решение	Граница утверждения
Язык	Java	Предложен для будущей реализации
Интерфейс	JavaFX	Окна, формы, таблицы и состояния
Хранение	SQLite через JDBC и слой доступа	Локальная автономная версия
Email	Сервисный слой и SMTP	Пока только проектируемые события
Версионирование	Git и GitHub	Фактически использовано для материалов

6.1. Логическая модель данных

Сущность	Ключевые поля	Назначение
User	user_id, full_name, email, password_hash, role, is_active	Профиль и права

Ticket	ticket_id, title, description, category, priority, status, author_id, assignee_id, timestamps	Заявка и её состояние
TicketComment	comment_id, ticket_id, author_id, text, created_at	Комментарии и результат
TicketHistory	history_id, ticket_id, actor_id, event_type, old_value, new_value, created_at	Неизменяемая хронология
Notification	notification_id, ticket_id, recipient_id, event_type, delivery_status	Проектируемое событие уведомления

7. Спецификация требований SRS

SRS фиксирует функциональные, нефункциональные, интерфейсные требования и критерии приёмки. Ниже приведены ключевые требования, которые непосредственно проверяются по UML и прототипу.

ID	Требование	Приоритет
FR-004	Система определяет роль и применяет её к действиям	Must
FR-010	Успешная заявка получает уникальный номер и статус «Открыта»	Must
FR-017	Принятие фиксирует техника, время и переход в «В работе»	Must
FR-018	Техник добавляет результат и переводит заявку в «Решена»	Must
FR-019	Автор подтверждает, что решение устраняет проблему	Must
FR-020	Отказ с комментарием возвращает заявку в «В работе»	Must
FR-021	Закрытие доступно технику только после подтверждения	Must
FR-023	Переход хранится с автором, временем и значениями	Must
FR-029	Создание и смена статуса формируют проектируемое email-событие	Should
FR-031	Будущая SMTP-интеграция может отправлять подготовленные события	Could

Критерий приёмки	Проверка
AC-03	Корректная форма создаёт уникальную заявку со статусом «Открыта»
AC-06	Приём фиксирует техника, время и статус «В работе»
AC-07	Статус «Решена» невозможен без результата
AC-08	Автор подтверждает решение или возвращает заявку с комментарием
AC-09	Закрытие доступно только после подтверждения
AC-11	Email-событие создаётся при создании и изменении статуса, но не трактуется как доставка

8. UML-модель

Use Case Diagram показывает границы системы и права двух акторов. Activity Diagram показывает поток от создания заявки до закрытия, включая проверку данных, решение, подтверждение, отказ и повторную обработку.

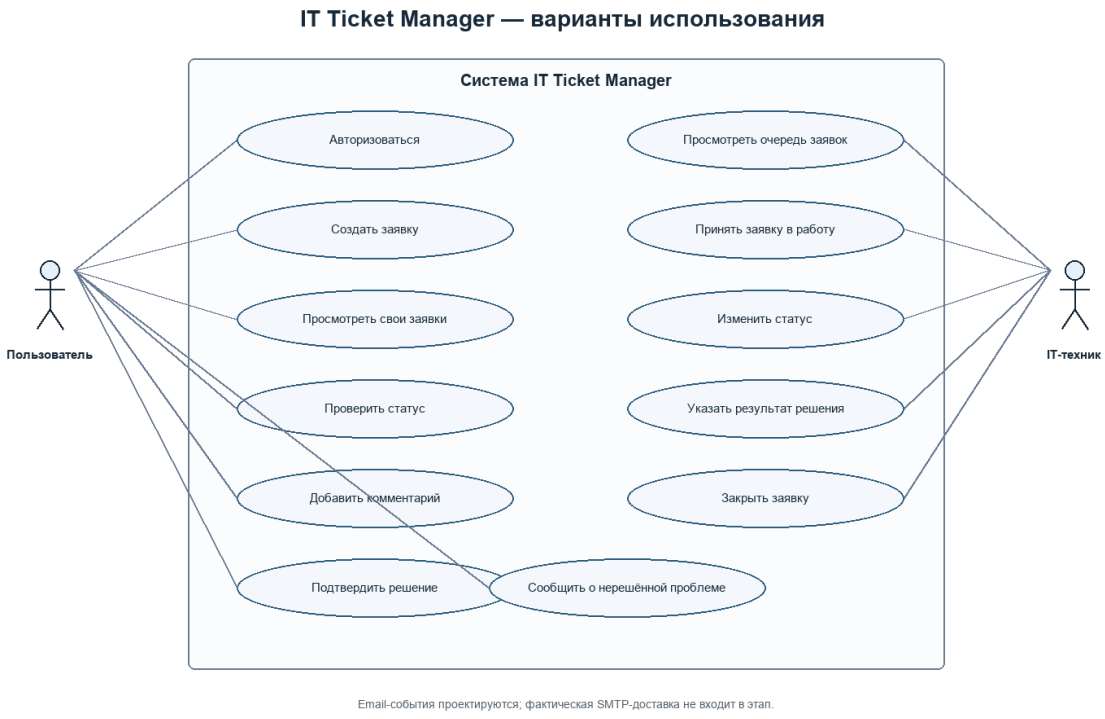


Рисунок 1 - Диаграмма вариантов использования

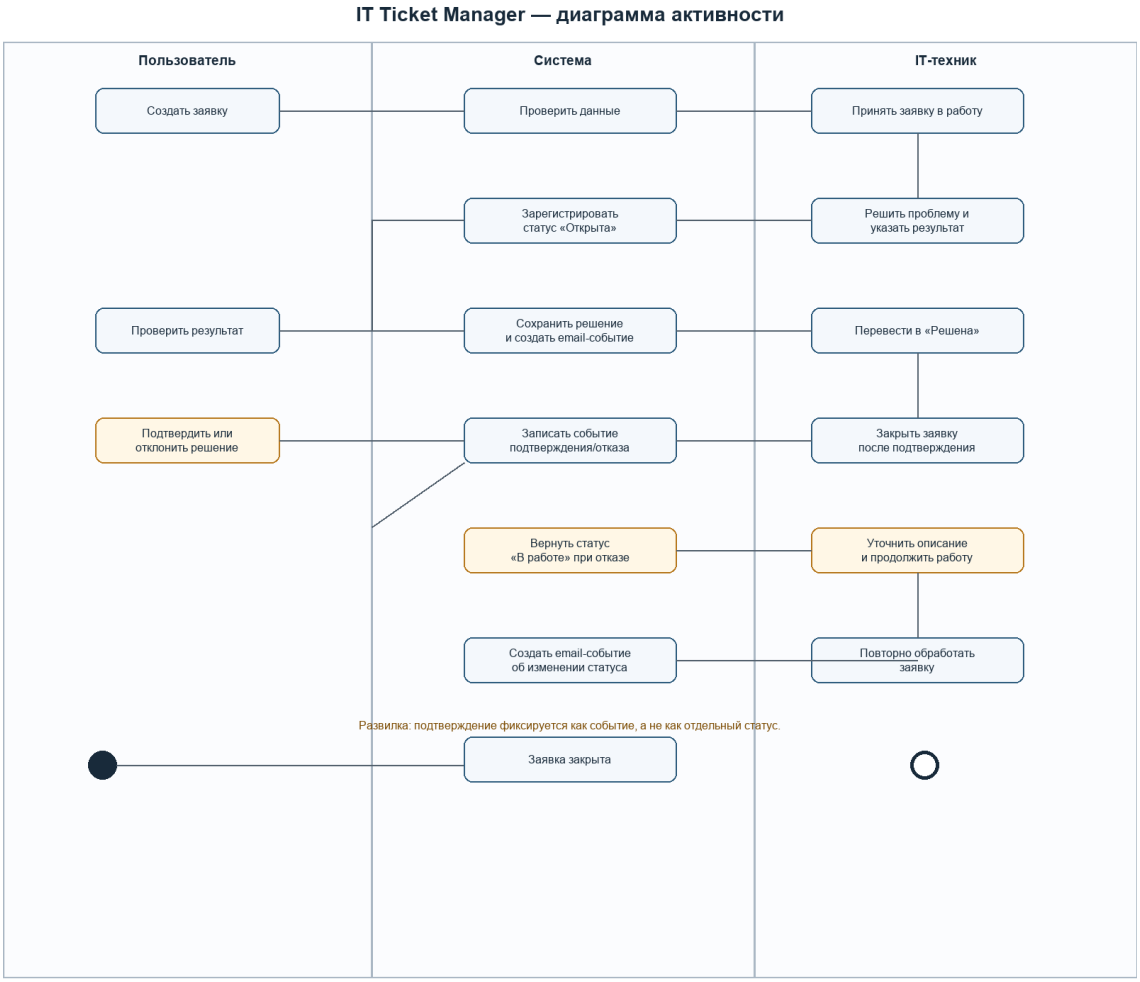


Рисунок 2 - Диаграмма активности

8.1. Исходник Use Case Diagram

```
@startuml
title IT Ticket Manager - Диаграмма вариантов использования
left to right direction
actor "Пользователь" as User
actor "IT-техник" as Tech
rectangle "Система IT Ticket Manager" {
  usecase "Авторизоваться" as U01
  usecase "Создать заявку" as U02
  usecase "Просмотреть свои заявки" as U03
  usecase "Проверить статус" as U04
  usecase "Добавить комментарий" as U05
  usecase "Подтвердить решение" as U06
  usecase "Сообщить о нерешённой проблеме" as U07
  usecase "Просмотреть очередь заявок" as T01
```

```
  usecase "Принять заявку в работу" as T02
  usecase "Изменить статус" as T03
  usecase "Указать результат решения" as T04
  usecase "Закрыть заявку" as T05
}
User --> U01
User --> U02
User --> U03
User --> U04
User --> U05
User --> U06
User --> U07
Tech --> U01
Tech --> T01
Tech --> T02
Tech --> T03
Tech --> T04
Tech --> T05
note bottom of T05
Закрытие разрешено только после
подтверждения автора заявки.
end note
note right of U07
Отказ автора возвращает заявку
в состояние «В работе».
end note
@enduml
```

8.2. Исходник Activity Diagram

```
@startuml
title IT Ticket Manager - Диаграмма активности
|Пользователь|
start
:Создать заявку;
|Система|
:Проверить данные;
if (Данные корректны?) then (да)
  :Зарегистрировать заявку;
  :Установить статус «Открыта»;
else (нет)
  :Показать ошибки формы;
  |Пользователь|
  :Исправить данные;
  |Система|
  :Повторить проверку;
```

```
endif
|IT-техник|
:Принять заявку в работу;
:Решить проблему и указать результат;
|Система|
:Сохранить решение;
:Установить статус «Решена»;
:Создать проектируемое email-событие;
|Пользователь|
:Проверить результат;
if (Решение подтверждено?) then (да)
 |Система|
 :Записать событие подтверждения;
 |IT-техник|
 :Закрыть заявку;
 |Система|
 :Создать email-событие о закрытии;
 stop
else (нет)
 :Указать причину отказа;
 |Система|
 :Записать событие отказа;
 :Вернуть статус «В работе»;
 :Создать email-событие;
 |IT-техник|
 :Уточнить данные и повторно обработать;
 |Пользователь|
 :Проверить результат повторно;
endif
@enduml
```

9. Прототип интерфейса в Figma

Прототип является визуальной и интерактивной моделью будущего интерфейса, а не реализованной программой. Редактируемый файл: https://www.figma.com/design/2uKVzo9PKUNHL4QuH4SUPk

Экран	Назначение
01 Авторизация	Вход пользователя или IT-техника
02 Главная панель	Список заявок и быстрый доступ к созданию
03 Создание заявки	Тема, описание, приоритет и условное вложение
04 Карточка заявки	Реквизиты, статус, результат и действия
05 Панель техника	Очередь, принятие в работу и решение

06 Подтверждение	Подтверждение результата или возврат в работу

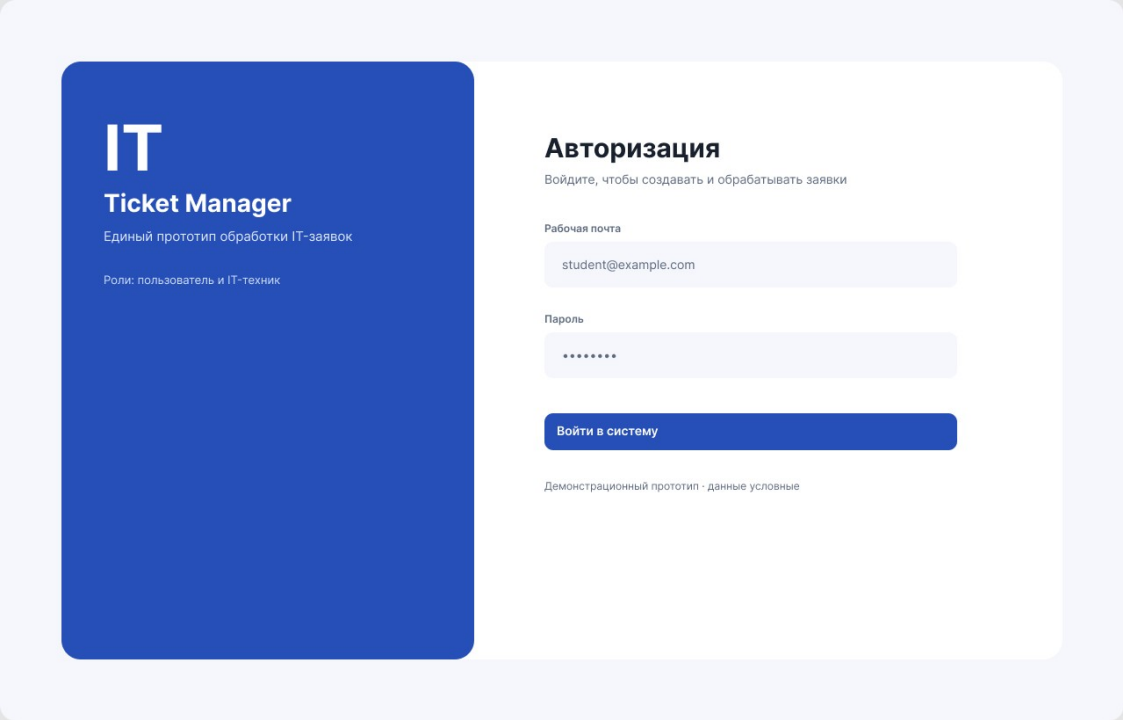


Рисунок 3 - Авторизация

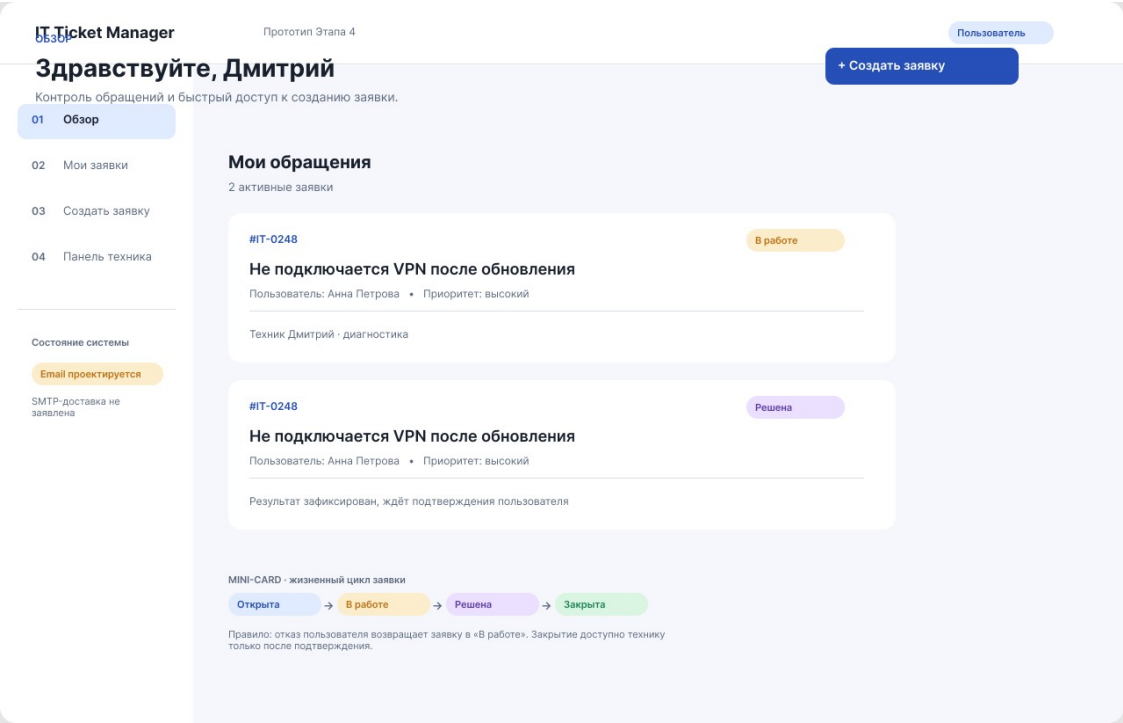


Рисунок 4 - Главная панель

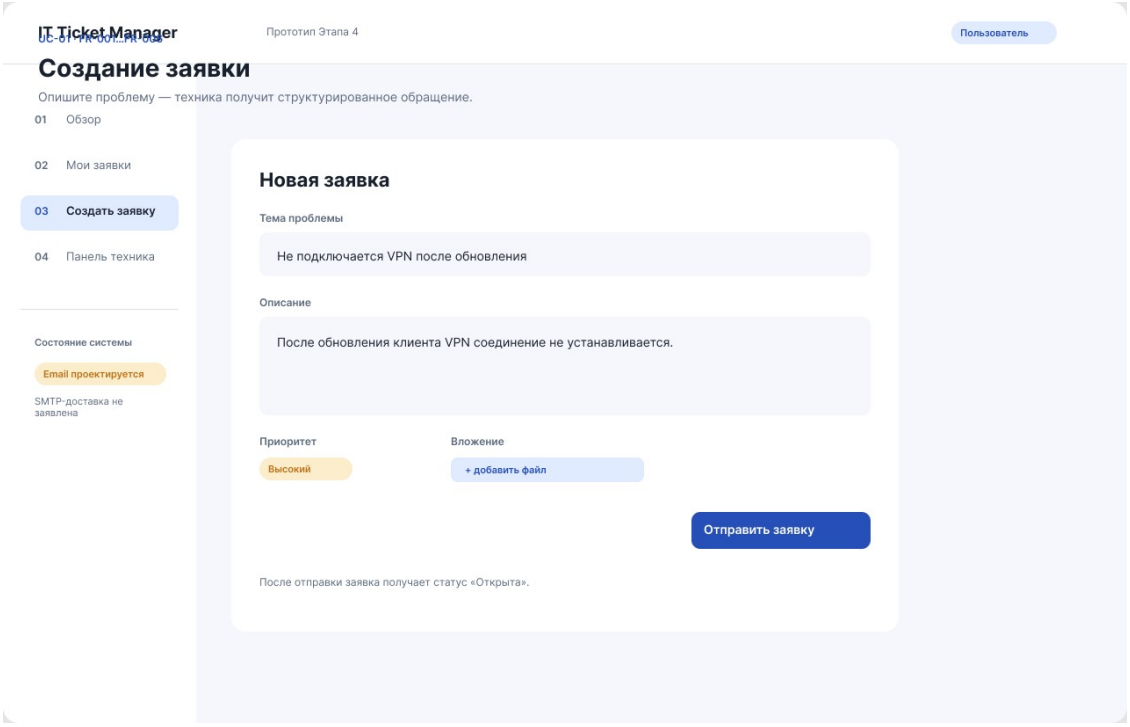


Рисунок 5 - Создание заявки

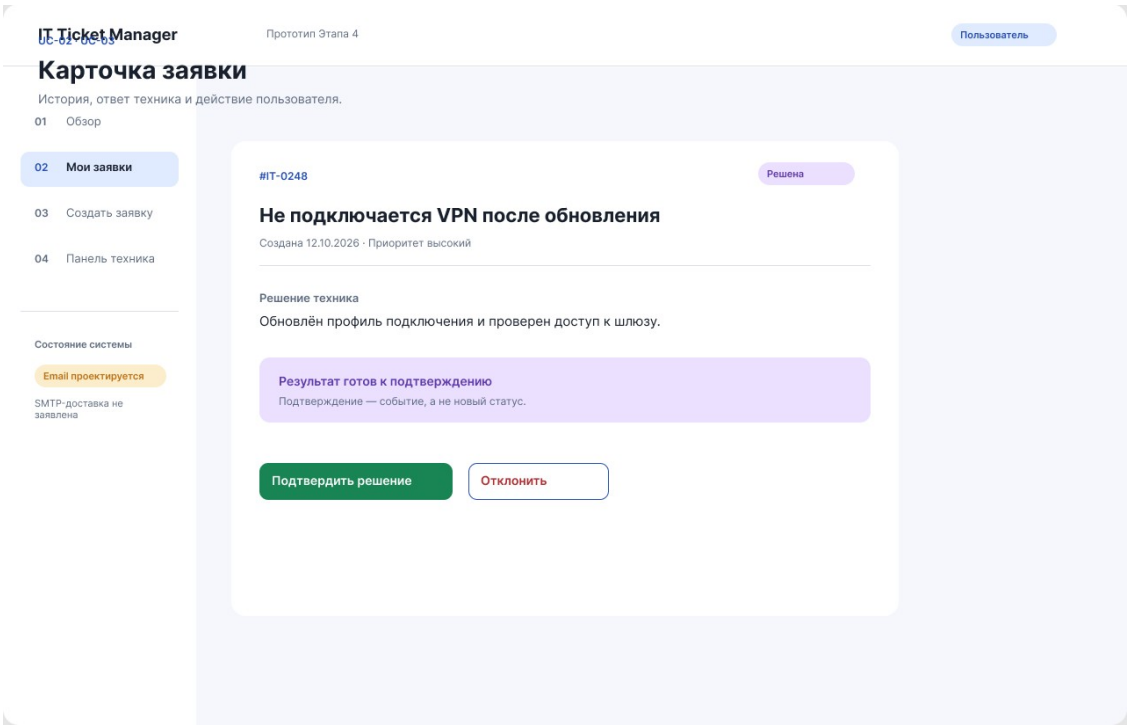


Рисунок 6 - Карточка заявки

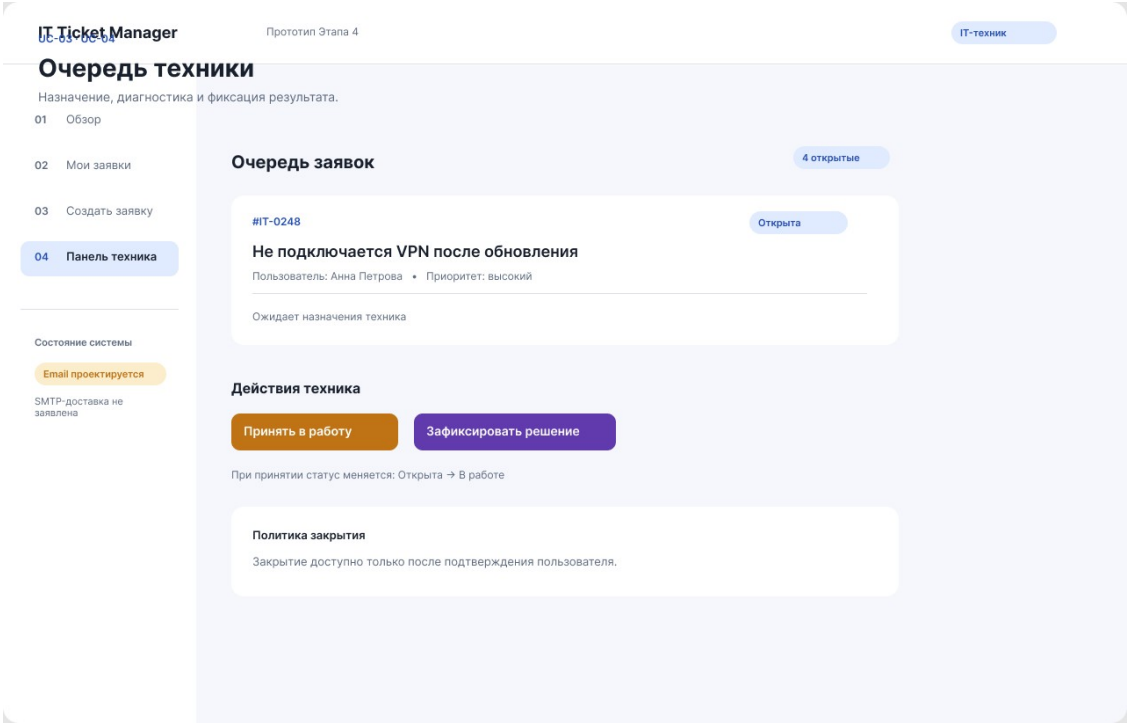


Рисунок 7 - Панель техника

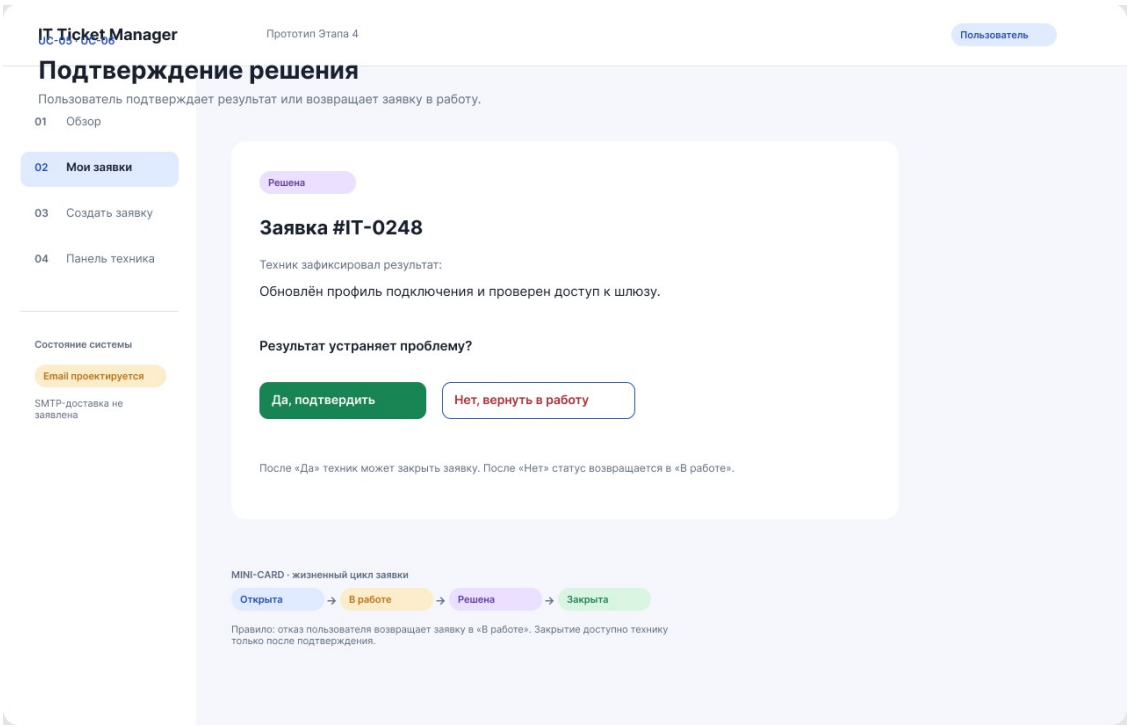


Рисунок 8 - Подтверждение решения

На экранах отражена mini-card со статусами «Открыта -> В работе -> Решена -> Закрыта». Подтверждение показано как событие, а отказ возвращает заявку в работу. Email обозначен только как проектируемая возможность.

10. Git и GitHub

Материалы этапов 1-5 организованы в локальном Git-репозитории и опубликованы в публичном репозитории IT-Ticket-Manager.

Параметр	Проверенный результат
GitHub	https://github.com/vertex34212/IT-Ticket-Manager
Ветка	main
История	3 коммита
Последний коммит	7ef5261 docs: record GitHub publication
Публикация	Выполнен push; локальная main синхронизирована с origin/main
Состав	FINAL, README, .gitignore, Git_Commands.md и проектные материалы

10.1. Основные Git-команды

```
cd /Users/dmitriit/Documents/ChatGPT/aplicatia
git status --short
git add .
git commit -m "docs: record GitHub publication"
git remote -v
git push -u origin main
```

Проверка публикации ограничена уже зафиксированным публичным результатом. В этом этапе новые изменения в GitHub автоматически не отправлялись: перед синхронизацией обновлённого отчёта требуется отдельное согласование публикации.

11. Проверка согласованности задания

Пункт задания	Статус	Подтверждение
Титульник и реквизиты	Выполнено	Студент, группа, преподаватель, дисциплина, вариант
Содержание и описание приложения	Выполнено	Разделы 1-6
Цели, пользователи, модули	Выполнено	Разделы 2-4
Java, JavaFX, SQLite	Выполнено	Раздел 6, только как планируемый стек
SRS	Выполнено	Раздел 7, ключевые FR/NFR/AC
UML и PlantUML	Выполнено	Раздел 8, диаграммы и исходники
Figma и скриншоты	Выполнено	Раздел 9, шесть экранов и ссылка

Git и GitHub	Выполнено	Раздел 10, 3 коммита и последний 7ef5261
Статусы и закрытие	Проверено	Закрытие только после подтверждения; отказ возвращает в работу
Email	Проверено	Только проектируемая функция, SMTP не выдан за факт
Оригиналы	Сохранены	Исходные материалы этапов 1-5 не удалялись

11.1. Чеклист качества

- содержание отчёта соответствует структуре этапов 1-5;
- роли ограничены Пользователем и IT-техником;
- статус «Подтверждена» не добавлен; подтверждение оформлено событием;
- закрытие разрешено только после подтверждения автора;
- отказ с причиной возвращает заявку в «В работе»;
- email упоминается только как проектируемое событие/будущая SMTP-интеграция;
- UML, SRS и Figma используют одинаковые статусы и роли;
- DOCX и PDF отрендерированы и визуально проверены;
- обновлён README и добавлен чеклист;
- публикация новых изменений в GitHub оставлена на согласование.

12. Заключение

В результате подготовлен единый итоговый отчёт по IT Ticket Manager. Он объединяет проектную идею, цели, роли, модули, требования SRS, технологический план, UML-модель, визуальный прототип и результаты работы с Git/GitHub. Основные бизнес-правила согласованы: заявка проходит путь «Открыта -> В работе -> Решена -> Закрыта», подтверждение пользователя является событием, отказ возвращает заявку в работу, а закрытие выполняется техником только после подтверждения.

Следующий практический шаг после согласования - реализация JavaFX-приложения с локальным SQLite, покрытие переходов тестами и отдельная проверка email-интеграции. Эти действия относятся к будущей разработке и не являются частью заявленных результатов этапа 6.

13. Проверенные источники

Источник	Использование
Этап 1: планирование	Назначение, цель, модули, роли, архитектура и план
Этап 2: SRS	FR/NFR/UI, сущности, переходы и критерии приёмки
Этап 3: UML	Use Case, Activity, PlantUML и трассировка
Этап 4: Figma	Ссылка на файл, карта экранов и правила

	прототипирования
Этап 5: GitHub	Git-команды, публичный репозиторий и история коммитов
GitHub repository	https://github.com/vertex34212/IT-Ticket-Manager
Figma file	https://www.figma.com/design/2uKVzo9PKUNHL4QuH4SUPk

Примечание: перечисленные источники являются локальными материалами проекта и указанными публичными ссылками. В отчёте не используются вымышленные публикации, неподтверждённые реализации или несуществующие интеграции.